

Software Design Document (SDD) – StudyQuest

Titel des Dokuments:

Software Design Document – StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.3

Datum:

27.02.2026

Author:innen:

- Michael Steer (Scrum Master)
- Luke Engehardt (Product Owner)
- Giuliana Carrano (Developer)
- Paul Strasser (Developer)
- Roman Faber (Developer)

Betreuer / Prüfer:

Sascha Wanningner

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbeschreibung
0.1	20.01.2026	G. Carrano	Initiale Gliederung und Modulübersicht entworfen
0.2	25.01.2026	G. Carrano	Datenmodell und Schnittstellen-Signaturen ergänzt
1.0	30.01.2026	G. Carrano	Erstfassung des SDD, basiert auf Requirements und Architektur-Dokumenten

Version	Datum	Autor	Änderungsbeschreibung
1.1	30.01.2026	G. Carrano	Formalia ergänzt: Kopfblatt, Autoren, Distribution List
1.2	12.02.2026	M. Steer	Algorithmen-Abschnitt (XP-Vergabe, Level-Up) überarbeitet
1.3	27.02.2026	G. Carrano	Konsistenzprüfung mit Analyseklassenmodell, Finalversion zur Abgabe vorbereitet

Distribution List

Name	Rolle	Kommentar / Zuständigkeit
Sascha Wanninger	Prüfer / Betreuer	Bewertung im Rahmen des Moduls
Michael Steer	Scrum Master	Koordination & Freigabe
Luke Engehardt	Product Owner	Anforderungen, Dokumentation & Technische Dokumentation
Giuliana Carrano	Developer	Projektskizze, Dokumentation, Architektur & UML
Paul Strasser	Developer	
Roman Faber	Developer	

1. Zweck & Geltungsbereich

1.1 Zweck dieses Dokuments

Dieses Dokument verbindet die Requirements mit konkreten Implementierungsdetails und Code-Referenzen. Der Fokus liegt auf: - **API-Schnittstellen** zwischen den Modulen - **Algorithmen und Geschäftslogik** (z. B. XP-Vergabe, Level-Up-Berechnung) - **Nicht-funktionale Anforderungen** (Performance, Security, Input-Validation) - **Testkonzepte** und Qualitätssicherung - **Direkte Code-Mappings** zu `src/js`

1.2 Abgrenzung zur Softwarearchitektur

Die Softwarearchitektur-Dokumentation behandelt die strukturelle Lösung des Problems: - Schichtmodell und Architektur-Pattern (3-Schichten, MVC, SPA) - Abhängigkeiten zwischen Analyse-Klassen und Technologien - Referenzarchitekturen und Design-Entscheidungen - Konzeptionelle Übersicht ohne Code-Referenzen

2. Module & Verantwortlichkeiten

(Übersicht, Implementierung in `src/js`)

- `index.html / css/` – UI-Templates, Styles.
- `ui.js` – DOM-Rendering, View-Updates, Toasts und Timer-Display.
- `app.js` – Router / Controller; Initialisierung, Seitenwechsel, Session-Management.
- `auth.js` – Registrierung, Login, Logout, „Passwort vergessen“-Flow (prototypisch).
- `user.js` – User-Model und Session-Logik (Level, XP-Aggregation, Rollen).
- `quest.js` – Quest-Lifecycle: `startQuest()`, `completeQuest()`, Timer-Schnittstellen.
- `grade.js` – CRUD für Noten, CSV Import/Export.
- `achievement.js` – Badge-Checks und Vergabe.
- `notification.js` – Erzeugung und Anzeige von Notifications.
- `leaderboard.js` – Ranking- und Streak-Berechnung.
- `admin.js` – Admin-Funktionen (Quest-Katalog, Regeln).
- `db.js` – Abstraktion der Persistenz (LocalStorage-Wrapper).

Hinweis: Jedes Modul exportiert global/namespaced Funktionen (kein bundler/ESM aktuell).

2.1 Entwurfsmuster (v1.0: Direkte Modulaufrufe statt Observer)

In der aktuellen Implementierung sind direkte Modulaufrufe verwendet: `quest.js` ruft `NotificationModel`, `AchievementSystem` und `UserModel` direkt auf (kein EventBus). Das Observer-Pattern wird als **Refactor-Vorschlag für v2.0** dokumentiert. Details, zukünftige Architektur und Nutzen des Patterns siehe:

`Documents/Softwarearchitektur/DesignPattern_Observer.md`.

3. Datenmodell (Domänenobjekte)

Kurzbeschreibung der primären Stores (LocalStorage-Key):

- **User (users):** { `id`, `email`, `name`, `password_hash`, `is_admin` (Boolean), `xp` (aktuelles Level-XP), `level`, `total_xp_earned` (kumulativ), `active_quest_id`, `quest_history`: [], `current_streak`, `best_streak`, `last_activity_date`, `avatar`, `created_at`, `updated_at` }
- **Quest (quests):** { `id`, `title`, `description`, `difficulty`, `xp_reward`, `status` }
- **LearningSession (sessions):** { `id`, `user_id`, `quest_id`, `xp_earned`, `duration_seconds`, `completed_at` }
- **Grade (grades):** { `id`, `user_id`, `module_name`, `grade_value`, `semester`, `created_at` }
- **Achievement (achievements):** { `id`, `user_id`, `key`, `title`, `icon`, `description`, `unlocked_at` } (separater Store pro User, nicht Teil von User)
- **Notification (notifications):** { `id`, `user_id`, `type`, `message`, `data`, `is_read`, `created_at` }
- **GameRule (game_rules):** { `level_threshold`, `xp_per_minute_timer`, `max_level` }

- **Timer (timers)**: { id, user_id, quest_id, start_time, end_time, duration_seconds, active }
-

4. Schnittstellen & API (aus Sicht der Module)

Kern-Schnittstellen (Beispiele):

- `db.get(store, id) → Objekt` | `db.save(store, obj) → id`
- `auth.register(email, password, name) → { success, message }` | User-Erstellung
- `user.authenticate(email, password) → User Objekt` (wirft Error bei Fehler)
- `quest.startQuest(userId, questId) → { questId, title, description, xp_reward, difficulty, started_at }`
- `quest.startTimer(userId, questId) → Timer Objekt`
- `quest.stopTimer(userId) → { timer, xpEarned, timeBonus, durationSec, leveledUp, newLevel, newAchievements }`
- `leaderboard.getTop10(criteria) → Array`
- `grade.importCSV(userId, csvText) / grade.exportCSV(userId)`

Kontrollfluss (v1.0): UI → `app.js` (Router/Controller) → Business-Module (`quest.js` , `user.js` , `achievement.js` , `grade.js` , etc.) → `db.js` → `LocalStorage`. Bei Quest-Abschluss: `quest.stopTimer()` ruft direkt folgende Module nacheinander auf: 1. `NotificationModel.notifyQuestCompleted()` 2. `UserModel.addXP()` (prüft & triggert Level-Up Notification) 3. `AchievementSystem.checkAndUnlock()` (prüft & triggert Achievement Notifications) 4. `UserModel.updateStreak()`

Diese direkte Sequenz erfolgt synchron, **nicht asynchron über EventBus**. (Siehe Abschnitt 3.1)

5. Wichtige Algorithmen & Regeln

- **XP-Vergabe**: Basis XP der Quest (in Questdefinition) + Zeitbonus = `(duration_minutes * game_rules.xp_per_minute_timer)` . Implementiert in `quest.js` , Zeilen 85-100.
 - **Level-Up**: Berechnet als `Math.floor(total_xp_earned / level_threshold) + 1` . `user.addXP()` prüft automatisch und setzt neues Level. Auslöser für Achievement-Checks.
 - **Leaderboard**: Sort Primary Key ist `total_xp_earned` descending. Für alle Rankings: nach dem gewählten Kriterium (xp/level/quests/streak/achievements) sortierend. Tie-Breaker: **Nicht implementiert** in v1.0 (würde `last_activity_date` sein, ist aber optional).
 - **Streaks**: Gezählt in `days_in_row` basierend auf `LearningSession` -Einträgen. Update in `UserModel.updateStreak()` prüft ob letzte Aktivität heute oder gestern war.
-

6. Nicht-funktionale Anforderungen & Sicherheit

- **Performance:** Dashboard/Leaderboard laden ≤ 2 s (Ziel). Vermeidung teurer Sortieroperationen auf großen Collections.
- **Security (aktuell):** Passwörter sind prototypisch gespeichert; Empfehlung: bcrypt/Argon2 + serverseitige Authentifizierung für produktive Nutzung.
- **XSS & Input Validation:** Alle Benutzereingaben vor dem Rendern escapen; `ui.js` sicheres DOM-Update.
- **Datensicherheit:** LocalStorage ist clientseitig sichtbar → sensible Daten niemals im Klartext speichern.

Empfehlung: Für produktive Nutzung Backend mit sichere Auth-Methode, HTTPS, serverseitiger Persistenz.

7. Testkonzept

- **Unit Tests:** Modularer JS-Code in testbaren Funktionen; empfohlen: Jest/JS test runner (Refactoring zu ESM/Modules erleichtert Testability).
 - **Integration Tests:** Manual / automatisiert (z. B. Playwright) für UI-Flows: Registrierung, Quest-Start/Completion, CSV-Import.
 - **Testfälle (Auswahl):**
 - UC01: Registrierung mit neuer E-Mail → `users` wächst um 1.
 - UC04/05: Quest starten → Status `active`; Quest abschließen → XP erhöht, `sessions` enthält Eintrag.
 - UC10: CSV-Import fehlerhafte Datei → Fehler, keine Änderung an `grades`.
-

8. Deployment, Wartung & Weiterentwicklung

- **Refactor-Vorschläge:**
 - Trenne Module in ESM/TypeScript für bessere Typensicherheit und Tests.
 - Ersatz `LocalStorage` durch REST-API + DB (z. B. Firebase/Postgres) mit Migrationspfad.
 - Security-Upgrade: serverseitige Auth & Passwort-Hashing.
 - **Wartung:** Dokumentation im `Documents/` -Ordner aktuell halten und Code-Dokumentation (JSDoc) ergänzen.
-

9. Mapping: Use Cases → Code (Kurz)

- **UC01 (Register/Login)** → `auth.js`, `user.js`, `db.js`
- **UC04/05 (Quests)** → `quest.js`, `user.js`, `db.js`, `achievement.js`, `notification.js`
- **UC06 (Timer)** → `quest.js`, `timers` -Store, `ui.js`

- **UC07/10 (Grades & CSV)** → `grade.js`, `db.js`
 - **UC11/12 (Achievements/Leaderboard)** → `achievement.js`, `leaderboard.js`, `db.js`
 - **Admin-UCs** → `admin.js`, `game_rules` -Store
-